Konzeption und prototypische Umsetzung eines webbasierten Tools zur Bearbeitung von IPTC-Daten

Praxisprojekt
im Rahmen des Bachelorstudiums *(B.SC)*
im Studiengang Informatik
an der Fakultät für Informatik und Ingenieurwissenschaften
der Technischen Hochschule Köln

vorgelegt von:	Felix Kilian Egbert
Matrikel-Nr.:	11125735
Adresse:	Wohlandstr. 9 51766 Engelskirchen felix.egbert@smail.th-koeln.de
eingereicht bei:	Prof. Christian Noss

Engelskirchen, 30.01.2026

Inhaltsverzeichnis

Abbildungsverzeichnis **III**

Glossar **IV**

Abkürzungsverzeichnis **V**

1. Einleitung **1**
1.1. Problemstellung und Kontext 1
1.2. Zielsetzung der Arbeit 1
1.3. Motivation 2

2. Grundlagen **3**
2.1. Digitale Bilddateien und Metadaten 3
2.2. Ursprung, Struktur und Bedeutung des IPTC Standards 3

3. Methodik **5**
3.1. Forschungsdesign und Vorgehensweise 5
3.2. Methoden und Verfahren 6
3.3. Technologien und Werkzeuge 7
3.3.1. Schreibwerkzeuge 7
3.3.2. Web Framework: Nuxt.js 4 mit Typescript 7
3.3.3. Clientseitige Metadatenverarbeitung 7
3.3.4. Speicherung: File System Access API und IndexedDB 8
3.3.5. Entwicklungswerkzeuge und Testautomatisierung 8
3.4. Systemarchitektur 9
3.4.1. Komponentenübersicht 9
3.4.2. Dateizugriff und Speicherarchitektur 9
3.4.3. Datenhaltung im Browser 9

4. Implementation **10**
4.1. Gesamtüberblick der Systemimplementierung 10
4.2. Implementierung des Parser-Moduls 11
4.2.1. Einlesen und Analyse des APP13-Segments 12
4.2.2. Dekodierung und Mapping der IPTC-DataSets 14
4.2.3. Serialisierung von IPTC-Daten in JPEG 14

4.2.4. Validierung der Parserfunktionalität mittels automatisierter Tests 16
4.3. Implementierung der Webanwendung 16
4.3.1. SPA, SSR und lokale Datenverarbeitung 17
4.3.2. Dateizugriff und Kommunikation mit dem Parser 17
4.3.3. Datenhaltung im Browser 19
4.3.4. Struktur und Rolle der einzelnen Stores 20
4.3.5. Dynamische Darstellung von IPTC-Feldern 21
4.3.6. Validierung der Eingabefelder 22
4.3.7. UI-Struktur und Nutzerinteraktion 23
4.3.8. Speichervorgang 24
4.4. Deployment und Betrieb 24

5. Fazit **26**
5.1. Zielerreichung 26
5.2. Reflexion der Umsetzung 26
5.3. Technologische Einordnung 26
5.4. Grenzen des aktuellen Prototyps 27
5.5. Ausblick / Weiterentwicklung 27

Literatur **28**

A. Anhang **31**
A.1. Code-Repository URL 31
A.2. Quellcode: *extractIIMBlockFromJPEG* 31
A.3. Quellcode: *searchSegmentForImageResourceBlocks* 32
A.4. Quellcode: *parseIIM* 32
A.5. Quellcode: *writeToJPEG* 34
A.6. Quellcode: *Dockerfile* 37

Abbildungsverzeichnis

3.1. Browser Unterstützung: File System Access API 8

4.1. Vereinfachte Systemarchitektur der Anwendung 11
4.2. Typisches JPEG APP13-Segment mit IPTC-Daten 13
4.3. Darstellung einer Kategorie in der Applikation 21
4.4. Abbildung: Screenshot der entwickelten Webanwendung 24

Glossar

Big Endian Eine Methode zur Speicherung von Multibyte-Daten, bei der das höchstwertige Byte an der niedrigsten Speicheradresse abgelegt wird. 12, 13, 15

Blob Ein Datentyp zur Speicherung unverständlicher Datenmengen als einheitliches Objekt. 8, 18

Composable Eine wiederverwendbare Funktion auf Basis der Vue Composition API, mit der sich zustandsbehaftete Logik kapseln und komponentenübergreifend nutzen lässt. 9, 10, 19, 20

ExifTool Ein plattformunabhängiges Kommandozeilen-Tool zur Analyse und Bearbeitung von Metadaten in Mediendateien. Es unterstützt zahlreiche Formate wie EXIF, IPTC und XMP (Baker, 2013). 6, 8, 11, 15, 16

Nuxt.js Ein auf Vue basierendes Meta-Framework für serverseitiges Rendering, modulare Entwicklung und performante Webanwendungen. 7, 10, 16, 17, 19, 26

Parser Ein Programm, welches z.B. Metadaten in einer Datei erkennt, ausließt und in eine strukturierte, für Anwendungen lesbare Form überführt. 2, 7, 9–12, 14, 16, 20, 26, 27

Tailwind CSS Ein Utility-First-CSS-Framework zur schnellen Gestaltung von Oberflächen direkt im Markup (Inc., 2026). 7

Toast Ein kleines Popup welches Feedback zu einer ausgeführten Operation enthält. 24

Vue.js Ein progressives JavaScript-Framework zur Erstellung komponentenbasierter Benutzeroberflächen im Web. 10, 17, 19, 20, 26

Abkürzungsverzeichnis

CI Continuous Integration. 16

DSR Design Science Research. 5

EXIF Exchangeable Image File Format. 1, 3, 27

IDB IndexedDB. 8–10, 19, 20, 26

IIM Information Interchange Model. 1–7, 9, 11–16, 18, 20–22, 26, 27

IPTC International Press Telecommunications Council. 1–3, 5–7, 9, 11, 12, 14–16, 18, 19, 22, 26, 27

IRB Image Resource Block. 4, 12, 13, 15

npm Node Package Manager. 8–11, 15, 18, 27

SPA Single Page Application. 17, 23

SSR Server-side Rendering. 17

XMP Extensible Metadata Platform. 1, 3, 4, 27

1. Einleitung

1.1. Problemstellung und Kontext

Metadaten spielen eine zentrale Rolle in der Arbeit mit digitalen Bilddateien. Sie ermöglichen es, beschreibende Informationen eines Bildes wie Titel, Urheberrecht oder Erstellungsort sowohl für Menschen als auch für Maschinen leserlich abzuspeichern. Über die letzten Jahrzehnte wurden verschiedene Standards entwickelt, die definieren, wie Metadaten in oder an Bilddateien angehängt oder abgespeichert werden. Einer dieser Standards ist der *IPTC Photo Metadata Standard*, der vom International Press Telecommunications Council (IPTC) entwickelt wurde und durch seine große Nutzung von Nachrichtenagenturen, Fotografen, Fotoagenturen, Bibliotheken, Museen und anderen relevanten Industrien als einer der meistgenutzten Standards für Bilddateien bekannt ist (International Press Telecommunications Council, 2025c).

Trotz seiner weiten Verbreitung ist der praktische Umgang mit IPTC-Metadaten in mancher Hinsicht schwierig. Vor allem der ältere technische IPTC-Standard Information Interchange Model (IIM) ist in modernen Softwarelösungen meist weniger bis gar nicht vertreten. Da die meisten heutigen Programme sich eher auf neuere Standards wie Extensible Metadata Platform (XMP) oder Exchangeable Image File Format (EXIF) fokussieren, gibt es viele Agenturen, Fotografen o.ä., deren Arbeit immer noch vom IIM-Standard abhängig ist. Besonders im webbasierten Bereich fehlt es an geeigneten Werkzeugen, die ein vollständiges Einsehen und Bearbeiten von IPTC-Metadaten, auch im IIM-Format, ermöglichen.

1.2. Zielsetzung der Arbeit

Das Ziel dieser Arbeit ist es, ein Konzept für ein webbasiertes Werkzeug zu entwickeln und prototypisch umzusetzen, das es ermöglicht, Bild-Metadaten im IPTC-IIM-Standard direkt im Browser anzuzeigen und zu bearbeiten. Die Anwendung soll plattformunabhängig und ohne zusätzliche Softwareinstallation nutzbar sein, um eine so einfach wie mögliche Benutzererfahrung zu ermöglichen.

Im Mittelpunkt steht einerseits die Entwicklung eines benutzerfreundlichen Interfaces, welches einen einfachen Umgang mit den verschiedenen Metadaten-Feldern

einer Bilddatei ermöglicht und dem Nutzer hilft, diese besser zu verstehen. Andererseits soll ein eigener Parser erstellt werden, welcher das Einlesen und Schreiben der IPTC-IIM-Metadaten ermöglicht. Dieser soll dann von der Webapplikation genutzt werden, um dort hochgeladene Bilder einzulesen und mit den eingegebenen Daten zu überschreiben.

Die Arbeit soll zur Modernisierung von Metadatenverwaltungssystemen beitragen. Sie zeigt auf, wie auch etablierte, aber technisch anspruchsvolle Standards in moderne, leicht zugängliche und kollaborative Systeme überführt werden können, ohne dass dabei Datenintegrität oder Standardkonformität verloren geht.

1.3. Motivation

Die Wahl des Themas für mein Praxisprojekt wurde von mehreren persönlichen und fachlichen Interessen geleitet. Dieses wollte ich unbedingt im Bereich der Webentwicklung umsetzen, da dieser schon seit mehreren Jahren der zentrale Schwerpunkt meiner Ausbildung und persönlichen Weiterentwicklung ist.

Die konkrete Idee zu dieser Arbeit stammt von Prof. Christian Noss. Auf meiner Suche nach einem Professor, welcher Projekte im Bereich Webentwicklung betreut, bin ich auf mehrere interessante Ausschreibungen für Projektideen auf seiner Webseite gestoßen. Hier hat mich vor allem das Thema eines webbasierten Editors für IPTC-IIM-Metadaten angesprochen, da ich selbst Interesse an Fotografie habe und mir die Verbindung zwischen Webtechnologie und der Bearbeitung von Bildmetadaten nicht nur fachlich spannend, sondern auch sehr praxisrelevant erschien.

Obwohl ich bereits umfassende Erfahrung im Bereich der Webentwicklung sammeln konnte, stellte die Kombination mit der technischen Verarbeitung von Bilddateien und Metadatenstrukturen direkt im Browser eine neue Herausforderung dar. Diese sah ich als Chance, mein Wissen in mir bislang weniger bekannten Konzepten zu erweitern.

Darüber hinaus bot das Projekt die Möglichkeit, theoretisches Wissen aus vorangegangenen Lehrveranstaltungen wie Mensch-Computer-Interaktion praktisch anzuwenden, um einen benutzerfreundlichen, webbasierten Prototyp zur Anzeige und Bearbeitung von IPTC-IIM-Metadaten zu konzipieren und umzusetzen.

2. Grundlagen

2.1. Digitale Bilddateien und Metadaten

Metadaten sind strukturierte Informationen, die wir erstellen, speichern und teilen, um Dinge zu beschreiben und deren Nutzung zu ermöglichen. Sie bilden die Grundlage dafür, wie wir Objekte identifizieren, interpretieren und miteinander verknüpfen und tragen maßgeblich dazu bei, das Wissen zu erschließen, das wir in Systemen suchen. Sie sind entscheidend für die Funktionalität von Systemen, die Inhalte verwalten, wie Suchmaschinen wie Google, Online-Shops wie Amazon, Social Media Plattformen wie Instagram, aber ebenso in Bibliotheken, Archiven oder Museen (Riley, 2017).

Auch bei digitalen Bildern übernehmen Metadaten diese zentrale Rolle. Sie enthalten ergänzende Informationen zum Bildinhalt, wie etwa das Aufnahmedatum, die geografischen Koordinaten, Urheberangaben, Titel, Beschreibung oder Lizenzhinweise. Da es heute möglich ist, eine virtuell unbegrenzte Anzahl an Bildern auf Handys oder Rechnern zu speichern, ist es umso wichtiger, einen Weg zu haben, diese effektiv durchsuchen zu können (Tesic, 2005).

Metadaten können entweder in einer separaten Datei gespeichert oder direkt in die Bilddatei eingebettet werden. Die Einbettung erfolgt dabei über standardisierte Formate, wobei EXIF, IPTC und XMP die heute am meisten genutzten Standards sind. (Smith et al., 2014).

2.2. Ursprung, Struktur und Bedeutung des IPTC Standards

Der International Press Telecommunications Council (IPTC) entwickelte 1990 mit dem Information Interchange Model (IIM) einen binär codierten Standard zur Speicherung redaktioneller Metadaten, welcher eine standardisierte, maschinenlesbare Struktur schaffen sollte. Besonders geeignet war er für Nachrichtenübertragungssysteme der damaligen Zeit, da er geringe Speicher- und Rechenressourcen benötigte (Tesic, 2005). Seine Anwendung in der Fotografie fand der Standard ein paar Jahre später, als die Softwarefirma Adobe Systems einen Teil der IIM-Eigenschaften in ihre eigene

Bildbearbeitungssoftware Photoshop übernahm und die Daten direkt in JPEG-Dateien in sogenannten „Image Resource Block (IRB)“ speicherte (Adobe Systems Incorporated, 2010).

Im Jahr 2001 stellte Adobe mit XMP ein modernes, XML-basiertes Framework vor, das mehr Flexibilität, Erweiterbarkeit und Interoperabilität bot. Zusammen mit dem IPTC wurde daraufhin das „IPTC Core Schema for XMP“ entwickelt, das die IIM-Strukturen in das neue XMP-Format überführte. 2007 wurde zusätzlich dazu das „IPTC Extension Schema“ entwickelt, welches die Anforderungen verschiedener Bereiche der professionellen Fotografie berücksichtigte. Beide Spezifikationen wurden 2008 erstmals als ein einheitliches Dokument unter dem Titel „IPTC Photo Metadata Standard“ veröffentlicht und werden seither regelmäßig aktualisiert. Zum Zeitpunkt dieser Arbeit fand die letzte Aktualisierung im Jahr 2024 statt (IPTC Photo Metadata Working Group, 2024).

3. Methodik

Die Methodik beschreibt das systematische Vorgehen bei der Konzeption und prototypischen Umsetzung des webbasierten Tools zur Bearbeitung von IPTC-IIM-Metadaten. Ziel ist es, die angewandten Forschungsansätze, technischen Entscheidungen und Werkzeuge transparent darzustellen.

3.1. Forschungsdesign und Vorgehensweise

Die vorliegende Arbeit verfolgt einen praxisorientierten, designbasierten Forschungsansatz (Design Science Research (DSR)), der auf die Lösung eines realen Problems durch die Entwicklung eines innovativen Artefakts - in diesem Fall eines webbasierten Tools - abzielt. Der DSR-Ansatz eignet sich besonders für technologische Fragestellungen, bei denen sowohl konzeptionelle als auch implementierungstechnische Aspekte berücksichtigt werden müssen (Vom Brocke et al., 2020).

Das Forschungsdesign orientiert sich methodisch an den Prinzipien der agilen Softwareentwicklung. Dieser iterative und inkrementelle Entwicklungsansatz stellt Flexibilität, kontinuierliche Verbesserung sowie eine enge Rückkopplung mit potenziellen Nutzer:innen in den Mittelpunkt (Sommerville, 2016). Die agile Vorgehensweise wurde gewählt, um eine rasche Reaktion auf sich verändernde Anforderungen im Entwicklungsprozess zu ermöglichen - insbesondere im Hinblick auf die Nutzerinteraktion mit der Software, die Kompatibilität mit bestehenden IPTC-IIM-Metadatenstandards und die technologischen Rahmenbedingungen im Webumfeld.

Das zentrale Element der agilen Methodik ist die Aufteilung des Entwicklungsprozesses in kurze, strukturierte Arbeitszyklen, sogenannte Sprints, in denen jeweils konkrete Teilfunktionen des Tools geplant, umgesetzt und getestet werden. Die kontinuierliche Validierung der Anforderungen erfolgt durch regelmäßige Reviews und Anpassungen des Backlogs, wodurch eine hohe Anpassungsfähigkeit gegenüber neuen Erkenntnissen und Feedback ermöglicht wird. Ergänzt wird dieser Ansatz durch Test-Driven Development (TDD), wodurch bereits im Entwicklungsprozess die technische Qualität und Wartbarkeit des Codes gefördert werden (Sommerville, 2016). Diese agile Struktur hat sich in der vorliegenden Arbeit als effektiv erwiesen, um sowohl technische als auch konzeptionelle Herausforderungen iterativ und benutzerzentriert zu lösen.

Für die Evaluation des Prototyps wurde besonders auf Usability, Performance und Standardkonformität mit dem IPTC-IIM-Format geachtet. Außerdem wurden verschiedene Testnutzer eingesetzt, um zu verhindern, dass sie sich an potenzielle Schwachstellen der Applikation gewöhnen und ihren Umgang mit dieser an die gefundenen Schwachstellen anpassen, um diese zu umgehen.

3.2. Methoden und Verfahren

Im Rahmen dieser Arbeit wurde ein methodischer Ansatz gewählt, der sich auf das Zusammenspiel von Prototyping, agiler Softwareentwicklung und technischer Anforderungsanalyse fokussiert. Ziel war es, ein lauffähiges, iterativ entwickeltes Webtool zu konzipieren, das die Lesbarkeit und Bearbeitbarkeit von IPTC-IIM-Metadaten für Endnutzer:innen vereinfacht. Die verwendeten Methoden entstammen dabei Praktiken der Softwaretechnik und der Mensch-Computer-Interaktion-Forschung (MCI), um technische Funktionalität mit anwendungsorientierter Gestaltung zu verbinden.

Ein zentrales methodisches Element war die Anwendung von evolutionärem Prototyping, bei welchem auf Basis früher funktionaler Kernkomponenten fortlaufend weitere Funktionen ergänzt werden. Dieses Vorgehen ermöglichte frühzeitiges Testen und Feedback durch Testnutzer:innen, was insbesondere bei der Implementierung der Benutzeroberfläche zur Bearbeitung und Darstellung von IPTC-IIM-Daten entscheidend war (Boehm, 1988). Durch den Einsatz agiler Prinzipien - insbesondere die Orientierung an Agilen Sprints - konnte flexibel auf technologische und konzeptionelle Herausforderungen reagiert werden.

Darüber hinaus wurde ein methodischer Fokus auf die spezifikationsgestützte Anforderungsanalyse gelegt, um die Kompatibilität mit dem IPTC-IIM-Standard zu gewährleisten. Die technischen Spezifikationen des IPTC bildeten die Grundlage für die Definition der Datenstrukturen, Eingabeformate und Validierungslogiken. Ergänzend wurde zur Verifikation der Metadaten das externe Open-Source-Tool ExifTool als Referenzwerkzeug genutzt (Baker, 2013).

Die Auswahl dieser Methoden basiert auf ihrer bewährten Effektivität in Softwareentwicklungsprojekten mit hohem Iterationsgrad und geringer Planbarkeit zu Projektbeginn (Schatten, 2010). Die iterative Vorgehensweise ermöglichte ein sukzessives Verfeinern sowohl der technischen Architektur als auch der Benutzerführung des Tools.

3.3. Technologien und Werkzeuge

Für die prototypische Umsetzung des webbasierten Tools zur Bearbeitung von IPTC-IIM-Metadaten kamen moderne Webtechnologien sowie spezialisierte Werkzeuge zum Einsatz, die eine performante, vollständig clientseitige Verarbeitung digitaler Bilddaten ermöglichen. Die eingesetzten Technologien wurden hinsichtlich ihrer Eignung für modulare, testbare und standardkonforme Webapplikationen ausgewählt.

3.3.1. Schreibwerkzeuge

Für die Erstellung dieser Arbeit wurde primär Overleaf verwendet - ein cloudbasierter LaTeX-Editor mit plattformunabhängigem Zugriff und automatischer Versionssicherung. Ergänzend kamen KI-gestützte Werkzeuge zum Einsatz, um die sprachliche Qualität der Arbeit zu optimieren. Dabei wurden sowohl der KI-Assistent von Overleaf als auch das Tool ChatGPT von OpenAI verwendet, um Stil, Grammatik und Formulierungen zu verbessern. Der Einsatz dieser Tools diente ausschließlich der sprachlichen Optimierung, nicht der inhaltlichen Generierung. Auf diese Weise konnte eine präzise, konsistente und stilistisch angemessene Ausarbeitung sichergestellt werden.

3.3.2. Web Framework: Nuxt.js 4 mit Typescript

Die Anwendung wurde mit Nuxt.js 4 und TypeScript umgesetzt. Der Einsatz von Nuxt ermöglichte eine modulare, effiziente Entwicklung und bot durch bestehende Vorerfahrungen einen schnellen Einstieg. Die Wahl fiel auch im Hinblick auf mögliche zukünftige Erweiterungen um serverseitige Funktionen, die Nuxt dank hybrider Rendering-Modelle und nativer API-Unterstützung ohne Technologiewechsel erlaubt.

Für die Benutzeroberfläche kam die auf Tailwind CSS basierende Komponentenbibliothek NuxtUI zum Einsatz. Diese bot eine breite Auswahl an zugänglichen, anpassbaren UI-Komponenten an, wodurch sich sowohl der Entwicklungsaufwand als auch die gestalterische Komplexität reduzieren ließen.

3.3.3. Clientseitige Metadatenverarbeitung

Ein zentrales Merkmal der Anwendung ist die vollständige clientseitige Verarbeitung der Metadaten. Dafür wird ein eigener Parser als modulares Paket entwickelt, das direkt auf die Binärstruktur von JPEG-Dateien zugreift und darin eingebettete Metadaten ausliest oder verändert. Das Modul ist lokal eingebunden und wird in

	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Internet	WebView Android	WebView on iOS	Deno
showOpenFilePicker	✓ 86	✓ 86	✗ No	✓ 72	✗ No	✓ 132	✗ No	✓ 87	✗ No	✓ 29	✓ 132	✗ No	✗ No

Abbildung 3.1.: Browser Unterstützung: File System Access API

einer Monorepo-Struktur verwaltet. Zur Sicherstellung der Korrektheit wurden die Ergebnisse regelmäßig mit ExifTool abgeglichen (Baker, 2013).

3.3.4. Speicherung: File System Access API und IndexedDB

Zur lokalen Dateiverwaltung wurde die File System Access API genutzt, die - nach Nutzerfreigabe - direktes Lesen und Schreiben im Dateisystem ermöglicht. Da diese API jedoch noch nicht in jedem Browser verfügbar ist (siehe Abbildung 3.1), kommt als Fallback das npm-Paket `browser-fs-access` zum Einsatz. Es erkennt fehlende Unterstützung und ersetzt die Funktionalität durch klassische Datei-Dialoge und Blob-basierte Downloads.

Für das Speichern der getätigten Veränderungen wird der aktuelle Status ebenfalls in der IndexedDB (IDB) gespeichert, um ein konsistentes Nutzererlebnis zu ermöglichen. Somit werden die Veränderungen zwischen-gespeichert und sind selbst nach dem schließen und Wiederöffnen des Browsers weiterhin vorhanden.

3.3.5. Entwicklungswerkzeuge und Testautomatisierung

Die Entwicklung erfolgte in Visual Studio Code, unterstützt durch ESLint zur Einhaltung konsistenter Formatierung und grundlegender Clean-Code-Prinzipien.

Für die Versionsverwaltung wurde Git in Kombination mit GitHub genutzt. Ein strukturierter Workflow mit separaten Feature-Branches und Pull Requests sorgte für Übersichtlichkeit. Alle Änderungen wurden per Squash-Merge integriert, nachdem automatisierte Tests erfolgreich durchlaufen waren.

Diese Tests wurden über GitHub Actions automatisiert. Bei jedem Commit wurden Linting sowie Unit-Tests des Parsers ausgeführt. Ergänzend fanden manuelle UI-Tests statt, um die Funktionalität aus Nutzersicht sicherzustellen.

3.4. Systemarchitektur

Die entwickelte Anwendung basiert auf einer vollständig clientseitigen Architektur, die es ermöglicht, Bilddateien direkt im Browser zu laden, auszulesen, zu bearbeiten und zu speichern - ohne jegliche Serverkommunikation. Grundlage dafür sind moderne Webstandards wie die File System Access API und die Nutzung der IDB für die persistente Datenspeicherung.

3.4.1. Komponentenübersicht

Das System gliedert sich in zwei Hauptkomponenten: einen Parser zur Verarbeitung der IPTC-IIM-Metadaten und eine Webanwendung zur Anzeige und Bearbeitung dieser Daten. Beide Komponenten liegen zusammen in einem Monorepo, und der Parser wird mithilfe von pnpm workspaces in die Webanwendung eingebunden.

3.4.2. Dateizugriff und Speicherarchitektur

Der Zugriff auf lokale Bilddateien erfolgt mithilfe der File System Access API, die nach expliziter Nutzerfreigabe den direkten Lese- und Schreibzugriff auf das lokale Dateisystem innerhalb des Browsers ermöglicht. Dies ist allerdings derzeit nur in Chromium-basierten Browsern wie Chrome und Edge möglich. Um die Cross-Browser-Kompatibilität sicherzustellen, wurde zusätzlich das npm-Paket `browser-fs-access` integriert. Dieses abstrahiert den Zugriff und nutzt bei inkompatiblen Browsern automatisch die klassische Kombination aus `input` und Dateidownload.

3.4.3. Datenhaltung im Browser

Die Anwendung speichert sämtliche Daten lokal im Browser, ohne externe Server oder Cloud-Anbindung. Die persistente Speicherung erfolgt über mehrere logisch getrennte Speicherbereiche, sogenannte „Object Stores“ in der IDB, die über Composables in die Applikation angebunden sind. Diese Struktur ermöglicht eine klare Trennung von Zuständigkeiten und sorgt für Übersichtlichkeit sowie Konsistenz im Zusammenspiel von Datenhaltung und Benutzeroberfläche.

4. Implementation

Im folgenden Kapitel wird die konkrete technische Umsetzung der Anwendung detailliert beschrieben. Dabei wird die Architektur in ihre zentralen Teilbereiche zerlegt und die jeweiligen Komponenten hinsichtlich ihrer Funktion, Technologieauswahl und Umsetzung erläutert. Ziel ist es, die praktische Anwendung der in der Methodik vorgestellten Konzepte transparent und reproduzierbar darzustellen.

Die vollständige Implementierung der in diesem Kapitel beschriebenen Applikation ist in einem separaten Code-Repository umgesetzt. Der Link zum Repository ist im Anhang A.1 aufgeführt.

4.1. Gesamtüberblick der Systemimplementierung

Die Anwendung besteht aus zwei eng zusammenhängenden Hauptkomponenten: einem Parser zur Verarbeitung von Metadaten und einer Webanwendung zur Nutzerinteraktion. Beide Komponenten wurden eigenständig entwickelt und sind über ein gemeinsames Monorepo strukturell verbunden. Die Implementierung folgt einem modularen Aufbau, wodurch eine klare Trennung von Zuständigkeiten, hohe Wiederverwendbarkeit und einfache Testbarkeit gewährleistet sind (Tkachenko et al., 2025).

Das Parser-Modul ist als zustandsloses npm-Paket umgesetzt und in die App via `pnpm workspaces` eingebunden. Es liest Metadaten aus JPEG-Dateien, überführt sie in ein JSON-Format und kann modifizierte Daten wieder zurückschreiben.

Die Webanwendung wurde mit Nuxt.js 4 und TypeScript entwickelt und erlaubt die vollständige clientseitige Anzeige, Bearbeitung und Speicherung der Metadaten. Das Parser-Modul verarbeitet beim Laden und Speichern die Metadaten der JPEG-Dateien und gibt die aktualisierten Daten zurück.

Für den Bearbeitungszustand nutzt die App die Browser-interne IndexedDB (IDB), abstrahiert über das npm-Paket textttVueUse. Drei logisch getrennte Datenbereiche verwalten Datei-Infos, Auswahlstatus und Metadatenänderungen. Die Interaktion erfolgt über Vue.js Composables, die Funktionen für Lese-, Speicher- und Änderungsoperationen kapseln.

Eine vereinfachte Darstellung der Systemarchitektur ist in Abbildung 4.1 dargestellt.

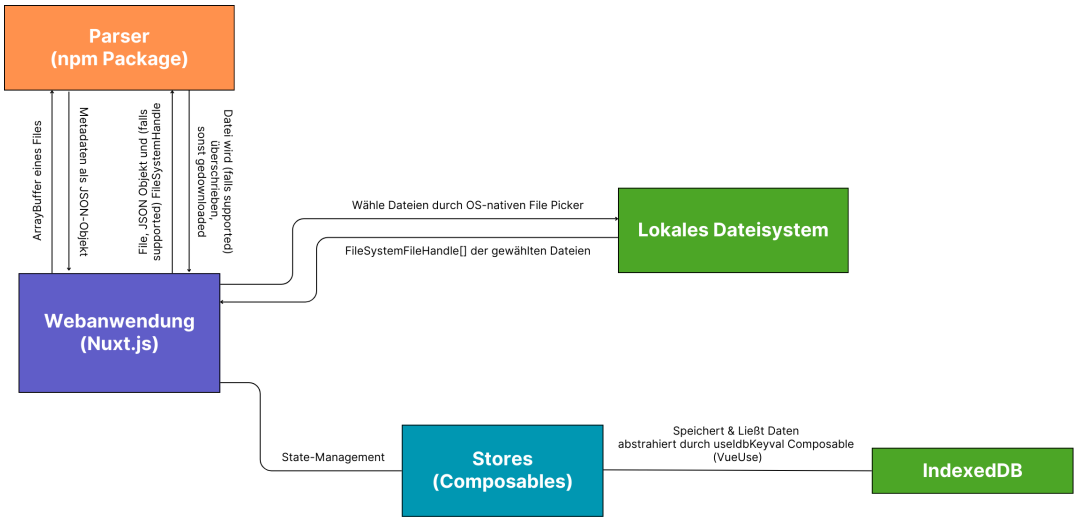

Abbildung 4.1.: Vereinfachte Systemarchitektur der Anwendung

4.2. Implementierung des Parser-Moduls

Das Parser-Modul bildet die technische Grundlage für das Einlesen und Schreiben von IPTC-IIM-Metadaten in JPEG-Dateien. Es wurde als zustandsloses npm-Paket konzipiert, das in der Webanwendung über ein Monorepo integriert ist, aber auch unabhängig einsetzbar bleibt. Die extrahierten Metadaten werden in eine JSON-Struktur überführt und lassen sich in modifizierter Form wieder in JPEG-Dateien einbetten.

Trotz bestehender Lösungen wie ExifTool fiel die Entscheidung zugunsten einer leichtgewichtigen, vollständig integrierten Eigenentwicklung ohne native Abhängigkeiten, mit Fokus auf dem IPTC-IIM-Standard.

Die Eigenentwicklung ermöglicht volle Kontrolle auf Byte-Ebene. Dies verbessert nicht nur die Genauigkeit und Erweitbarkeit, sondern fördert auch ein tieferes Verständnis der Spezifikation. Externe Tools abstrahieren diese Prozesse meist vollständig, wodurch wichtige Einblicke in die Struktur und Verarbeitung der Metadaten verloren gegangen wären.

Die Verarbeitung erfolgt ausschließlich mit nativen JavaScript-Funktionalitäten, ohne zusätzliche JPEG-Bibliotheken. Der Parser liest Metadaten durch Identifikation und Dekodierung relevanter Segmente und kann diese gezielt überschreiben oder neu anlegen. Die nächsten Abschnitte erläutern die technischen Schritte im Detail.

4.2.1. Einlesen und Analyse des APP13-Segments

JPEG-Dateien bestehen aus Marker-Segmenten, die verschiedene logische Abschnitte wie Bilddaten oder Metadaten kennzeichnen. Jeder Marker beginnt mit einem Präfix-Byte `0xFFF`, gefolgt von einem Typ-Byte (`0x01-0xFE`). Marker mit Nutzdaten (z.B. APP-Segmente) enthalten zusätzlich ein zwei Byte großes Längenfeld im Big Endian-Format sowie die eigentlichen Inhalte. Andere bekannte Marker sind etwa `0xFFD8` (Start of Image) oder `0xFFd9` (End of Image) (International Telecommunication Union, 1992).

Zur Speicherung applikationsspezifischer Daten nutzt JPEG die Marker `0xFFE0-0xFFEF` (APP0-APP15). Das APP13-Segment (`0xFFED`) wurde ursprünglich von Adobe für die Software Photoshop spezifiziert und wird seitdem weitgehend standardisiert zur Einbettung von IPTC-IIM-Metadaten genutzt (Adobe Systems Incorporated, 2020).

Ein typisches APP13-Segment, das IPTC-Daten enthält, ist in Abbildung 4.2 dargestellt. Es besteht aus folgenden Komponenten:

1. **APP13-Marker** (`0xFFED`)
2. **Längenfeld** (2 Bytes)
3. **Header-Signatur**: ASCII-Zeichenfolge `"Photoshop 3.0"` gefolgt von einem Nullbyte (`0x00`)
4. **Image Resource Block (IRB)**, bestehend aus:
   - Resource Identifier (`"8BIM"`)
   - Resource ID (`0x0404` für IPTC-Daten)
   - Länge und Inhalt der Ressource (IPTC-IIM-Datenblock)

Der Parser durchsucht den JPEG-Puffer sequenziell nach APP13-Markern. Wird einer erkannt, wird die Segmentlänge berechnet, der Block extrahiert und auf die „Photoshop-3.0“-Signatur geprüft. Ist diese vorhanden, beginnt die Suche nach IRBs.

Ein IRB ist eine modulare Binärstruktur zur Einbettung verschiedener Ressourcentypen (z.B. Farbprofile, Metadaten). Jeder Block enthält einen Identifikator, eine „Resource ID“, einen optionalen Namen, ein Längenfeld und den eigentlichen Nutzdatenbereich. Für IPTC-Daten ist die Resource ID `0x0404` und das Namensfeld ist leer.

Die Extraktion der IPTC-Daten erfolgt über eine zentrale Funktion *extractIIMBlock-FromJPEG* (siehe Anhang A.2). Diese durchläuft den Puffer Byte für Byte und prüft dabei jeweils zwei aufeinanderfolgende Bytes auf das Auftreten des APP13-Markers.

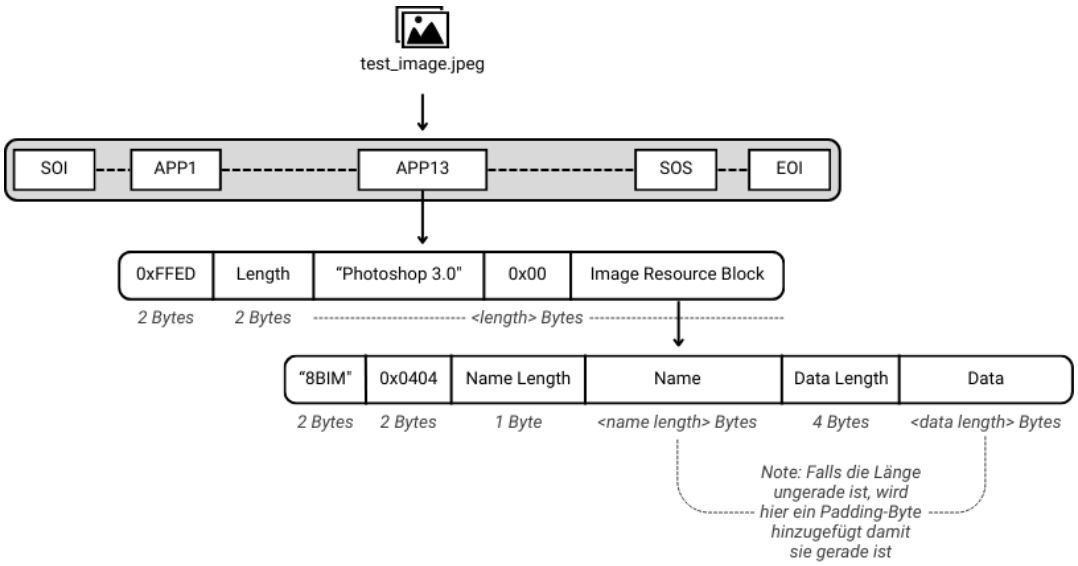

Abbildung 4.2.: Typisches JPEG APP13-Segment mit IPTC-Daten

Wird dieser gefunden, so wird anhand der folgenden zwei Bytes die Länge des Segments bestimmt. Dies erfolgt über eine eigene Hilfsfunktion `calculateSegmentLength`, welche die beiden Bytes unmittelbar nach dem Marker interpretiert. JPEG verwendet für Längenangaben das Big Endian-Format, sodass das erste Byte als High-Byte und das zweite Byte als Low-Byte gelesen wird. (Adobe Systems Incorporated, 2020) Um die Länge zu dekodieren, wird in der Implementierung ein `Bitshift` genutzt. Dies bedeutet, dass das erste Byte um acht Stellen nach links verschoben und anschließend mit dem zweiten Byte per logischem ODER kombiniert wird. Diese Methode entspricht einer direkten binären Zusammensetzung der beiden Bytes zu einem 16-Bit-Wert.

```
function calculateSegmentLength(buffer, offset) {
  const lengthStart = offset + 2
  return (buffer[lengthStart] << 8 | buffer[lengthStart + 1])
}
```

Wurde ein gültiges Segment gefunden, wird dieses daraufhin extrahiert und in der Funktion `searchSegmentForImageResourceBlocks` (siehe Anhang A.3) analysiert. Diese überprüft, ob das Segment einen IRB enthält, der der von Adobe spezifizierten Struktur für IIM-Daten entspricht (Adobe Systems Incorporated, 2020). Enthält es einen IIM-Block, wird dieser zurückgegeben.

4.2.2. Dekodierung und Mapping der IPTC-DataSets

Nach der Extraktion des IPTC-Blocks beginnt die logische Dekodierung der darin enthaltenen Datensätze. Jeder Datensatz folgt einer festen Struktur: Er beginnt mit dem Marker `0x1C`, gefolgt von einem Byte für die Record Nummer, einem Byte für die Datensatz Nummer, einem Längenfeld sowie der eigentlichen Nutzdaten, meist als UTF-8 oder ASCII-String kodiert.

Besonders zu beachten ist, dass das IIM zwei Varianten zur Angabe der Datenlänge kennt:

- Für Datensätze unterhalb einer Größe von 32.768 Bytes wird die Datenlänge in zwei Bytes angegeben.
- Überschreitet der Wert diese Grenze, gilt der Datensatz als „Extended“. Hier wird die Länge aus 5 Bytes zusammengesetzt, wobei das erste Byte immer den Wert `0x80` hat, gefolgt von vier weiteren Bytes, die gemeinsam die tatsächliche Länge kodieren (Comité International des Télécommunications de Press, 2014).

Die Kombination aus Record und DataSet ist standardisiert (z.B. `2:90`) und kann mehrfach auftreten. Im Parser übernimmt die Funktion `parseIIM` (siehe Anhang A.4) die Dekodierung. Sie wird mit einem JPEG-Puffer aufgerufen und extrahiert intern mithilfe der Funktion `extractIIMBlockFromJPEG` (siehe Anhang A.2) das relevante Segment und analysiert es byteweise per `DataView`.

Wird ein `0x1C-Marker` erkannt, liest die Funktion Record, DataSet und Länge aus und dekodiert die folgenden Bytes mit einem `TextDecoder`. Die Werte werden als Key-Value-Paare gespeichert, wobei der Schlüssel im Format `record:dataset` vorliegt und die Werte als `String` oder bei Mehrfacheinträgen als `String[]` gespeichert sind.

Obwohl die Funktionalität zur Verarbeitung wiederholbarer Felder im Parser bereits implementiert ist, wird sie in der aktuellen Version der Webanwendung noch nicht unterstützt. Diese Entscheidung wurde aus zeitlichen Gründen getroffen, da eine saubere und nutzerfreundliche Umsetzung sowohl im Datenmodell als auch in der Benutzeroberfläche einen großen zusätzlichen Entwicklungsaufwand erfordert hätte. Eine Erweiterung um diese Funktion ist jedoch für zukünftige Versionen vorgesehen.

4.2.3. Serialisierung von IPTC-Daten in JPEG

Während das Lesen von IIM-Daten bestehende Datensätze extrahiert, erfordert das Schreiben die umgekehrte Richtung: Ein Metadaten-Objekt muss in ein gültiges binäres Format serialisiert und in ein korrekt aufgebautes APP13-Segment eingebettet werden.

Zunächst wird aus den Eingabedaten eine geordnete Sequenz von Datensätzen gemäß IPTC-Spezifikation konstruiert (Comité International des Télécommunications de Press, 2014). Diese werden in einen IRB mit dem Identifikator `"8BIM"` und der Resource ID `0x0404` eingebettet, welcher wiederum in ein APP13-Segment verpackt wird. Falls in der Datei bereits ein APP13-Segment vorhanden ist, wird es ersetzt; andernfalls wird es neu eingefügt.

Die Funktion `writeToJPEG` (siehe Anhang A.5) übernimmt diesen Prozess. Sie akzeptiert ein `File` oder `Uint8Array`, ein JavaScript-Objekt mit IIM-Feldern sowie optional entweder einen Pfad (für Node.js Anwendungen) oder einen `FileSystemFileHandle` (für die File System Access API). Für den Fall, dass diese nicht unterstützt wird, kann außerdem optional noch ein Name übergeben werden, den die neue Datei haben soll.

Jedes Feld wird dabei in ein `DataSet` serialisiert - inklusive Marker, `Record-Nummer` , `DataSet-Nummer` , Länge und Daten, wobei die Kodierung der Daten per `TextEncoder` erfolgt. Felder über 32.767 Bytes werden als `Extended DataSet` verarbeitet und erhalten eine 5-Byte-Längenangabe im Big Endian-Format.

Der finale IRB enthält die nötigen Header (inkl. optionalem Padding-Byte bei ungerader Länge (Tesic, 2005)) und wird in ein APP13-Segment überführt. Beim Einfügen wird die JPEG-Datei nach einem vorhandenen APP13-Segment durchsucht, welches gegebenenfalls ersetzt oder, da die Reihenfolge der APPn-Segmente nicht klar definiert ist, neu hinter den vorhandenen APP-Segmenten angefügt wird (Adobe Systems Incorporated, 2020).

Je nach Umgebung erfolgt das Speichern unterschiedlich:

- In einer Node.js-Anwendung wird der Buffer direkt über `writeFileSync` persistiert.
- In einer Browser-Anwendung wird, sofern verfügbar, über die File System Access API die angegebene Datei direkt überschrieben. Falls diese nicht verfügbar ist, da der vom Nutzer genutzte Browser sie noch nicht unterstützt, erfolgt ein Fallback über das `browser-fs-access` npm-Paket, das eine Download-Aktion auslöst.

Die gesamte Implementierung ist in modernen TypeScript-Funktionen organisiert, nutzt keine externen Bibliotheken zur Binärverarbeitung und bleibt dadurch kompakt und browserkompatibel. Dies passt zur Gesamtstrategie der Anwendung, möglichst leichtgewichtig und vollständig im JavaScript-Ökosystem und somit Plattformunabhängig integrierbar zu sein. Die Serialisierung der Metadaten ist vollständig binärkompatibel mit gängigen Metadaten-Analysetools wie Adobe Photoshop oder ExifTool und ermöglicht damit eine nahtlose Integration in bestehende Workflows.

4.2.4. Validierung der Parserfunktionalität mittels automatisierter Tests

Die Validierung des Parser-Moduls erfolgt durch automatisierte Tests, die mit dem JavaScript-Testframework *Vitest* umgesetzt wurde. Ziel der Tests war es, die Korrektheit sowohl beim Einlesen als auch beim Schreiben von IPTC-IIM-Metadaten in JPEG-Dateien sicherzustellen. Als Grundlage für die Validierung dienten exemplarische JPEG-Dateien, deren Metadaten zuvor mit dem etablierten Tool ExifTool extrahiert und verifiziert wurden (Baker, 2013). Dadurch konnte sichergestellt werden, dass die Ergebnisse des selbst entwickelten Parsers mit den branchenüblichen Standards übereinstimmen.

Ein Beispiel für diesen Ablauf:

```
const existingJpeg = fs.readFileSync('test.jpg')
const metaData = parseMetadata(existingJpeg)
metadata['2:90'] = 'Cologne'
writeMetadata(jpeg, metadata, 'test-out.jpg')
const updated = parseMetadata(fs.readFileSync('test-out.jpg'))
expect(updated['2:90']).toBe('Cologne')
```

Besonders hervorzuheben ist die zusätzliche Validierung der Extended DataSets. Dieser Sonderfall ist besonders wichtig zu testen, da er im praktischen Umfeld eher selten auftritt und somit Fehler schnell übersehen werden können.

Die automatisierten Tests wurden mit jedem Commit über eine Continuous Integration (CI)-Pipeline mithilfe von GitHub Actions ausgeführt, wodurch Regressionsfehler frühzeitig erkannt und systematisch vermieden werden konnten. Dieses Vorgehen folgt bewährten Prinzipien testgetriebener Entwicklung (Freeman und Pryce, 2009).

4.3. Implementierung der Webanwendung

Die prototypische Webanwendung dient als benutzerfreundliche, plattformunabhängige Schnittstelle zur Visualisierung, Bearbeitung und Speicherung von IPTC-IIM-Metadaten in JPEG-Dateien. Sie wurde mit dem Framework Nuxt.js 4 entwickelt und vollständig in TypeScript umgesetzt. Die gesamte Verarbeitung findet clientseitig statt, wobei sowohl der Parser zur Metadatenverarbeitung als auch die Datenhaltung direkt im Browser integriert sind.

4.3.1. SPA, SSR und lokale Datenverarbeitung

Für die Realisierung der Anwendung wurde das moderne JavaScript-Framework Nuxt.js 4 gewählt. Dieses ermöglicht die Entwicklung von Single Page Application (SPA) mit optionalem Server-side Rendering (SSR) und bietet dadurch ein flexibles Fundament sowohl für client- als auch serverseitige Webapplikationen. In der hier realisierten prototypischen Version wurde Server-side Rendering (SSR) aktiviert, jedoch nur zum initialen rendern der Webseite, da die gesamte Funktionalität ausschließlich im Browser ausgeführt wird, ohne dass serverseitige Logik zum Einsatz kommt.

Die Entscheidung, Nuxt.js gegenüber einem klassischen Vue.js-Setup zu bevorzugen, resultiert primär aus der langfristigen Planbarkeit: Der modulare Aufbau und die native Unterstützung für SSR und API-Routen erlauben es, die Anwendung künftig um serverseitige Funktionen zu erweitern, ohne grundlegende Architekturänderungen vorzunehmen. Die Nuxt.js-Architektur gilt als etabliert im modernen Webökosystem und wurde mehrfach als robuste Lösung für Webapplikationen identifiziert (Zaidan und Suyatno, 2025).

Ein weiterer zentraler Aspekt war die Entscheidung, sämtliche Verarbeitungsvorgänge lokal im Browser des Nutzers durchzuführen. Die gesamte Metadatenextraktion, -bearbeitung und -speicherung erfolgt somit ausschließlich auf dem Endgerät, wodurch ein Höchstmaß an Datensouveränität und -schutz gewährleistet werden kann. Besonders im Kontext von Bildmetadaten, welche etwa geographische Informationen oder Urheberangaben enthalten, ist eine rein lokale Verarbeitung aus datenschutzrechtlicher Sicht vorteilhaft.

4.3.2. Dateizugriff und Kommunikation mit dem Parser

Moderne Webapplikationen bieten zunehmend Funktionen, die traditionell nativen Anwendungen vorbehalten waren. Insbesondere der direkte Zugriff auf das lokale System, welcher in der vorliegenden Anwendung essenziell ist, da sämtliche Bilddateien vom Nutzer lokal geladen, verarbeitet und gespeichert werden, ohne dass eine Kommunikation mit einem externen Server stattfindet. Dieser Abschnitt beschreibt die eingesetzten Technologien zur Realisierung des Dateizugriffs sowie die damit verbundene Integration des entwickelten Parser-Moduls zur Extraktion und Modifikation der Metadaten.

Zugriff über die File System Access API

Die Applikation nutzt primär die File System Access API, eine Webstandard-API, die von Chromium basierten Browsern bereitgestellt wird. Diese API erlaubt es, mit echten Datei-Handles zu arbeiten und dabei sowohl Lese- als auch Schreiboperationen auf lokale Dateien mit persistenten Zugriffsrechten durchzuführen. Für den Nutzer bedeutet dies: Eine Bilddatei kann geöffnet, direkt bearbeitet und anschließend direkt gespeichert werden - ganz ohne Upload.

Die API ist aktuell nur in Chromium-basierten Browsern (wie Google Chrome oder Microsoft Edge) vollständig verfügbar und stellt daher einen technologischen Grenzfall zwischen Innovation und Kompatibilitätsproblemen dar.

Beispielhaft kann die Nutzung der API zur Dateiauswahl wie folgt aussehen:

```
const [fileHandle] = await window.showOpenFilePicker();
const file = await fileHandle.getFile();
const contents = await file.text();
textArea.value = contents;
```

Kompatibilitäts-Fallback: browser-fs-access

Da nicht alle Browser die File System Access API unterstützen, kommt zusätzlich das npm-Paket `browser-fs-access` zum Einsatz. Dieses wurde von Google Chrome Labs entwickelt und bietet eine elegante Fallback-Strategie: Es prüft zur Laufzeit, ob die File System Access API verfügbar ist, und verwendet andernfalls klassische `<input type="file»`-Dialoge sowie die Blob-Download-Methode zur Speicherung von Dateien. Dadurch wird sichergestellt, dass die Anwendung auch in nicht-chromium-basierten Browsern (z.B. Firefox oder Safari) nutzbar bleibt, wenn auch mit reduzierter Benutzerfreundlichkeit.

Kommunikation mit dem Parser-Modul

Sobald eine Bilddatei geladen wurde, wird sie zur Analyse an das interne Parser-Modul übergeben. Dieses ist als npm-Paket eingebunden und verarbeitet Bilddaten auf Byte-Ebene. Es bietet dafür zwei Hauptfunktionen:

- `parseMetadata` - zur Extraktion aller IPTC-IIM-Metadaten aus einer JPEG-Datei.
- `writeMetadata` - zum Überschreiben oder Hinzufügen von Metadaten.

Die Übergabe erfolgt über eine zentrale Steuerlogik innerhalb eines Vue.js Composables, die sicherstellt, dass das Parsen im Hintergrund geschieht. Beispielhaft zum Auslesen:

```
const buffer = await file.arrayBuffer()
const metadata = parseMetadata(new Uint8Array(buffer))
```

oder beim Speichern:

```
async function updateMetadata(file, metadata) {
    await writeMetadata(file.file, metadata, undefined, file.
        handle)
}
```

Die eigentliche Persistierung dieser Daten erfolgt im IDB-Speicher des Browsers. Dies wird im folgenden Abschnitt genauer erläutert.

4.3.3. Datenhaltung im Browser

Ein zentrales Gestaltungsmerkmal der entwickelten Webanwendung ist die vollständige Clientseitigkeit - sowohl im Hinblick auf die Verarbeitung der Bilddaten als auch auf deren temporäre Speicherung. Alle Metadaten, die aus einer Bilddatei extrahiert oder durch den Nutzer verändert werden, verbleiben ausschließlich im lokalen Kontext des Browsers. Diese Designentscheidung folgt dem Prinzip der `Privacy by Design` (Schaar, 2010), das insbesondere im Umgang mit personenbezogenen Daten eine hohe Relevanz besitzt, wie sie z.B. bei Urheberangaben, Ortsdaten oder Copyright-Informationen in IPTC-Daten auftreten können.

Zur Umsetzung der Datenhaltung wird die browserseitige Speichertechnologie IndexedDB (IDB) verwendet. Bei dieser handelt es sich um eine objektbasierte NoSQL-Datenbank, die persistenten Speicher in modernen Browsern bereitstellt. Über eine Abstraktion mit der Open-Source-Utility `vueuse/useIDBKeyval` wird eine reaktive Integration in das Vue.js/Nuxt.js-Framework ermöglicht, wodurch alle gespeicherten Daten automatisch mit der Benutzeroberfläche synchronisiert bleiben.

Verwendung von Stores zur Datenorganisation

In der Webanwendung werden Vue.js Composables genutzt, um wiederverwendbare Funktionalitäten zu kapseln. In Anlehnung an klassische Zustandscontainer wie Pinia wird in diesem Zusammenhang von „Stores“ gesprochen. Technisch handelt es sich jedoch nicht um reaktive Stores im klassischen Sinne, sondern um spezialisierte Composables, die gezielte Zugriffe auf die IDB ermöglichen.

Jeder Store (z.B. `useFiles`) ist als eigene Funktion implementiert und bündelt reaktive Variablen sowie zugehörige Operationen. Für die Verbindung zur IDB wird `VueUse/useIDBKeyval` verwendet. Diese Bibliothek integriert die IDB direkt in das Reaktivitätssystem von Vue.js, wodurch ein asynchroner, zustandsgetriebener Datenfluss entsteht, der eine automatische Synchronisation zwischen persistentem Speicher und Benutzeroberfläche ermöglicht.

Diese Architektur fördert eine klare Trennung von Logik und Darstellung, erhöht die Wiederverwendbarkeit und erleichtert Tests. Sie bildet damit eine wartbare und flexibel erweiterbare Grundlage für die Datenverarbeitung in der Anwendung.

4.3.4. Struktur und Rolle der einzelnen Stores

Jeder Store ist eine in einem Composable definierte, benannte Funktion (z.B. `useFiles`). Beim Aufruf stellt sie interne Refs (reaktive Datenstrukturen) sowie Methoden zur Verfügung, über die auf die jeweilige Datenstruktur zugegriffen wird.

In der Anwendung kommen drei voneinander getrennte Stores zum Einsatz, die unterschiedliche Aufgaben im Datenfluss übernehmen.

useFiles Enthält alle vom Nutzer geladenen JPEG-Dateien, einschließlich der zugehörigen Metadaten und `FileHandles`. Darüber hinaus wird hier das Lesen und Schreiben der Metadaten mithilfe des Parsers abgewickelt.

useFileSelection Verwaltet die aktuelle Dateiauswahl im File-Explorer und dient ausschließlich der UI-Steuerung. Die Auswahl wird mit einem reaktiven Objekt gespeichert und persistent abgelegt, sodass der Zustand auch nach einem Neuladen der Seite erhalten bleibt.

useFileState Beinhaltet den Bearbeitungszustand der Metadaten einer Datei. Für jede Datei wird ein Array aus allen editierbaren Feldern erzeugt. Grundlage dafür ist ein zentrales Mapping der unterstützten IIM-Felder, wodurch sichergestellt ist, dass nur gültige Eingaben im Frontend erscheinen.

Die Trennung in spezialisierte Stores sorgt für eine klare Struktur und erleichtert sowohl die Wartung als auch die Erweiterbarkeit der Anwendung. Jeder Store übernimmt eine eindeutig definierte Aufgabe, wodurch die Datenflüsse nachvollziehbar bleiben und Änderungen gezielt vorgenommen werden können.

Abbildung 4.3.: Darstellung einer Kategorie in der Applikation

4.3.5. Dynamische Darstellung von IPTC-Feldern

Ein zentrales Element der Benutzeroberfläche ist die dynamische Darstellung bearbeitbarer Metadaten. Statt für jedes Feld eine separate Eingabe zu definieren, generiert die Anwendung die Eingabefelder modular auf Grundlage eines zentralen Mappings. Dieses basiert auf der zum Zeitpunkt aktuellen IIM-Spezifikation (Version 4.2) (Comité International des Télécommunications de Press, 2014) und enthält für jedes Feld Informationen wie Schlüssel, Bezeichnung, Eingabetyp, Zeichensatz und maximale Länge.

Ein beispielhafter Mapping-Eintrag:

```
{
  key: '2:105',
  title: 'Headline',
  description: 'A publishable entry providing a synopsis of the
      contents of the objectdata.',
  octets: { max: 256 },
  allowedCharacterTypes: [CharacterTypes.graphic, CharacterTypes.
      space],
  type: 'text',
  mandatory: false,
  repeatable: false,
}
```

Zur besseren Orientierung werden die Felder im UI in thematische Kategorien gruppiert, die aus einem Titel und einer Array Struktur bestehen. Jede Zeile enthält ein oder mehrere Felder und kann optional Breitenangaben enthalten:

```
{
  title: 'Description & Textual Content',
  rows: [[{ key: '2:105', width: 30 }], [{ key: '2:120', width: 70
      }]],
},
```

Eine Darstellung der Kategorie in der Applikation wird in der Abbildung 4.3 gezeigt.

Die Darstellung erfolgt über typisierte Komponenten, wobei pro Eingabetyp (z.B. Textfeld, Auswahlfeld) spezialisierte Komponenten existieren. Für komplexere Felder wie Referenzen stehen eigene Komponenten mit zusätzlicher Logik bereit. Eine zentrale Wrapper-Komponente übernimmt die Auswahl und Einbindung der passenden Eingabekomponenten basierend auf dem Mapping.

Die Komponenten interagieren direkt mit den reaktiven Zuständen, welche sich im `useFileState` Store befinden, sowie mit der Validierungslogik. Die konsequente Trennung von Darstellung, Zustand und Logik folgt dem Prinzip der „Seperation of Concerns“ und sorgt für eine wartbare und erweiterbare Architektur (Kulkarni und Reddy, 2003).

Der dynamische Aufbau reduziert Redundanzen und vereinfacht die Pflege sowie die Erweiterung der Anwendung erheblich.

4.3.6. Validierung der Eingabefelder

Die Validierung der eingegebenen Metadaten erfolgt vollständig innerhalb der Webanwendung und ist eng an die dynamische Felddefinition geknüpft. Anstatt externe Validierungsbibliotheken zu verwenden, wurde ein eigenes System entwickelt, welches spezifische Regeln aus der IPTC-IIM-Spezifikation implementiert. Dadurch bleibt die Validierungslogik nah an der Datenstruktur und ist leicht wartbar und erweiterbar.

Die Regeln beinhalten:

Maximallänge Viele Felder des IPTC-IIM-Standards sind in ihrer Byte-Länge begrenzt. Diese Einschränkungen werden bei Texteingaben automatisch berücksichtigt und kommuniziert.

Zeichensatzbeschränkungen Für einige Felder sind nur bestimmte Zeichen erlaubt (z.B. Buchstaben, Zahlen oder Leerzeichen). Diese Einschränkungen werden über reguläre Ausdrücke umgesetzt.

Formatvalidierung Datums- oder Zeitfelder prüfen, ob die Eingabe dem geforderten Format entspricht.

Abhängigkeit zwischen Feldern Einige Felder erfordern konsistente Eingaben in anderen Feldern. So kann z.B. ein Zeitstempel nur gesetzt werden, wenn auch ein Datum angegeben ist. Solche Abhängigkeiten werden durch spezifische Validierungsfunktionen pro Feld umgesetzt.

Die Validierung findet in Echtzeit während der Eingabe statt. Nutzer erhalten bei ungültigen Werten visuelles Feedback (z.B. Farbcodierung, Tooltips oder Fehlermeldungen) und können Eingaben entsprechend korrigieren. Ist ein ungültiger Wert vorhanden, wird der Nutzer zudem daran gehindert, die Datei zu speichern. Dieses Vorgehen sichert die Zuverlässigkeit der erzeugten Metadaten.

4.3.7. UI-Struktur und Nutzerinteraktion

Die Benutzeroberfläche der Anwendung ist als Single Page Application (SPA) mit einem klar strukturierten Layout umgesetzt, das eine intuitive Navigation und effiziente Metadatenbearbeitung ermöglicht. Sie besteht aus zwei Hauptbereichen: dem Dateiexplorer (links) und dem dynamischen Metadaten-Editor (Hauptbereich).

Nach dem Import von JPEG-Dateien über den Upload-Button erscheinen diese als Liste im Explorer. Beim Auswählen einer Datei lädt der Editor automatisch die zugehörigen Felder.

Alle Eingabefelder werden dynamisch basierend auf dem zentralen Mapping-Objekt erzeugt. Je nach Typ (Text, Auswahl, Datum etc.) wird automatisch die passende Komponente gerendert, was die Oberfläche konsistent und leicht erweiterbar macht.

Besondere Interaktionsfunktionen umfassen:

Undo pro Feld Rücknahme einzelner Änderungen ohne Verlust des gesamten Bearbeitungsstands.

Reset pro Datei Wiederherstellung des ursprünglichen Zustands der Datei.

Echtzeit-Validierung Ungültige Eingaben werden sofort markiert und blockieren das Speichern.

Visuelles Änderungsfeedback Modifizierte Felder werden hervorgehoben.

Browserkompatibilitäts-Hinweis Bei fehlender Unterstützung der File System Access API wird der Nutzer informiert.

Diese Struktur ermöglicht auch technisch weniger versierten Nutzer:innen eine sichere und einfache Bearbeitung strukturierter Metadaten direkt im Browser.

Abbildung 4.4 zeigt die Anwendung beim Bearbeiten einer JPEG-Datei.

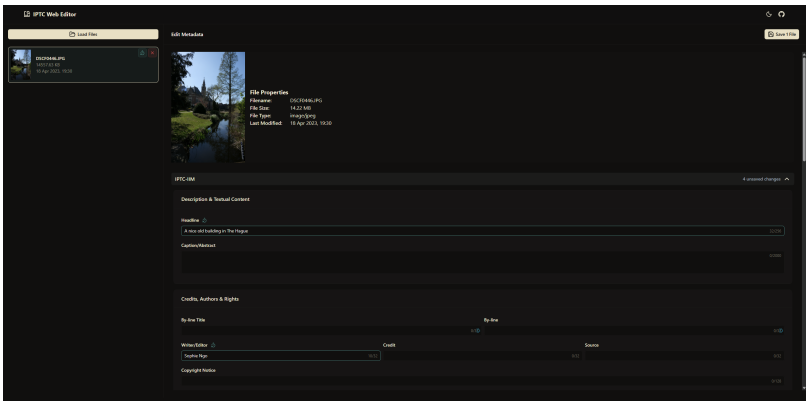

Abbildung 4.4.: Abbildung: Screenshot der entwickelten Webanwendung

4.3.8. Speichervorgang

Der Speichervorgang basiert auf einer klar getrennten Zustandsverwaltung: Änderungen werden zunächst im `useFileState` Store gehalten und beim Speichern mit den ursprünglichen Metadaten verglichen. Für jede geänderte Datei schreibt das Parser-Modul die aktualisierten Werte direkt in eine JPEG-Datei.

Je nach Umgebung erfolgt dies über verschiedene Wege. Ist die File System Access API vorhanden, wird die JPEG-Datei direkt per `FileHandle` überschrieben. Andernfalls wird eine Kopie der JPEG-Datei, welche die aktualisierten Metadaten enthält, heruntergeladen. In Node.js Umgebungen wird die Datei direkt am angegebenen Pfad gespeichert.

Nach dem Speichern wird der lokale Zustand synchronisiert und die Änderungen in der IndexedDB persistiert. Der Nutzer enthält eine Rückmeldung per Toast.

4.4. Deployment und Betrieb

Die Anwendung wird auf einem Virtual Private Server (VPS) von Hetzner betrieben, auf dem das Deployment-Tool Coolify installiert ist. Dieses automatisiert den Bereitstellungsprozess durch eine direkte Anbindung an das GitHub-Repository und erstellt bei jedem Commit in den Main-Branch automatisch einen Build. Die Auslieferung erfolgt über ein Docker-Image, das über ein eigenes Dockerfile definiert ist (siehe Anhang A.6).

Da es sich um eine rein clientseitige Anwendung handelt, läuft der Betrieb in einem einfachen Node.js-Container. Dank lokaler Datenverarbeitung im Browser bleiben die Anforderungen an den Server minimal, und die Anwendung bleibt auch bei langsamer Internetverbindung performant.

5. Fazit

5.1. Zielerreichung

Ziel dieser Arbeit war die prototypische Entwicklung einer vollständig webbasierten Anwendung zur Anzeige, Bearbeitung und Speicherung von IPTC-IIM-Metadaten in Bilddateien. Zu diesem Zweck wurden zwei zentrale Komponenten realisiert: ein eigener Parser zur Verarbeitung der Metadatenstruktur sowie eine Webanwendung auf Basis von Nuxt.js 4 zur Interaktion mit den Bilddateien im Browser. Die angestrebte Funktionalität, also das Lesen, Editieren und Speichern von Metadaten, konnte somit erfolgreich umgesetzt werden.

5.2. Reflexion der Umsetzung

Die Kombination aus eigener Parser-Implementierung und dynamischer Webanwendung erwies sich als tragfähig und funktional. Besonders gelungen war die enge Integration in das JavaScript-Ökosystem, wodurch eine nahtlose Kommunikation zwischen der Webanwendung und dem Parser ermöglicht wurde. Herausfordernd war vor allem die genaue Interpretation des proprietären Formats sowie die strukturierte Abbildung der Daten für die Benutzerschnittstelle und die korrekte reaktive Speicherung der Daten im Browser. Durch modulare Architekturelemente, insbesondere durch die Store-ähnlichen Vue.js Composables, gelang jedoch eine flexible und wartbare Struktur.

5.3. Technologische Einordnung

Die Wahl von Nuxt.js 4 und TypeScript bot eine moderne Grundlage für die Anwendungsentwicklung. Die clientseitige Datenverarbeitung mithilfe der File System Access API sowie der Nutzung von IndexedDB (IDB) ermöglichte eine performante und datenschutzfreundliche Lösung, die vollständig eigenständig operiert. Die Architektur profitiert dabei von einem klaren Komponentenmodell und reaktiven Datenflüssen, wodurch auch komplexere Interaktionen zuverlässig abgebildet werden konnten.

5.4. Grenzen des aktuellen Prototyps

Trotz des funktionalen Umfangs handelt es sich bei der aktuellen Umsetzung um einen Prototyp. Die Validierung der Daten erfolgt ausschließlich im Frontend, während der Parser keinerlei formale Prüfung durchführt, weshalb dieser momentan nur in Verbindung mit der App genutzt werden sollte. Auch wiederholbare Felder werden momentan nur Parser-seitig unterstützt und werden nicht von der Benutzeroberfläche unterstützt. Das Editieren von mehreren Dateien gleichzeitig war ursprünglich auch geplant, musste jedoch aus zeitlichen Gründen gestrichen werden.

5.5. Ausblick / Weiterentwicklung

Zukünftige Entwicklungen könnten sich zunächst auf die Generalisierung und Veröffentlichung des Parsers als eigenständiges npm-Paket konzentrieren. Dabei wäre insbesondere die Erweiterung um zusätzliche Metadatenstandards wie IPTC-Core und -Extension, XMP oder EXIF sowie die Integration einer formalen Validierung der Eingabedaten von zentraler Bedeutung.

Für die Webanwendung selbst bestehen verschiedene sinnvolle Erweiterungsmöglichkeiten. Ein naheliegender nächster Schritt wäre die Umsetzung wiederholbarer Felder, um den IPTC-IIM-Standard vollständig abzudecken. Anschließend könnten nutzerfreundliche Funktionen, wie eine mehrsprachige Benutzeroberfläche, eine Drag-and-Drop-Dateiverwaltung oder Verbesserungen in der Verständlichkeit der Felder, beispielsweise durch Beschreibungen, ergänzt werden.

Langfristig wäre die Integration fortgeschrittener Funktionalitäten denkbar. Dazu zählen beispielsweise eine kollaborative Bearbeitung von Metadaten in Echtzeit mittels WebSockets oder KI-gestützte Felder, etwa zur automatischen Generierung von Titeln oder Beschreibungen auf Basis des Bildinhalts.

Auf der Basis dieser Erweiterungen könnte perspektivisch auch ein kommerzielles Geschäftsmodell entwickelt werden. Komplexere Funktionen könnten hierbei im Rahmen eines Premium-Modells angeboten werden, während die Basisfunktionalität weiterhin offen zugänglich bliebe.

Literatur

Adobe Systems Incorporated. (2005). *A Manager's Introduction to Adobe eXtensible Metadata Platform, The Adobe XML Metadata Framework* (White paper). Adobe. https://www.osplabs.com/wp-content/uploads/whitepaper/xmp_whitepaper.pdf

Adobe Systems Incorporated. (2010). Adobe Photoshop File Formats Specification.

Adobe Systems Incorporated. (2020). *XMP Specification Part 3: Storage in Files* (Specification). Adobe. https://github.com/adobe/XMP-Toolkit-SDK/blob/main/docs/XMPSpecificationPart3.pdf

Baker, E. (2013). EXIF Custom: Automatic image metadata extraction for Scratchpads and Drupal. *Biodiversity Data Journal*, *1*, e973. https://doi.org/10.3897/BDJ.1.e973

Boehm, B. W. (1988). A spiral model of software development and enhancement. *Computer*, *21*(5), 61–72. https://doi.org/10.1109/2.59

Comité International des Télécommunications de Press. (2014, Juli). *IPTC - NAA Information Interchange Model Version 4* (Specification Nr. Version No. 4). IPTC. https://www.iptc.org/std/IIM/4.2/specification/IIMV4.2.pdf

Freeman, S., & Pryce, N. (2009, Oktober). *Growing Object-Oriented Software, Guided by Tests* [Google-Books-ID: QJA3dM8Uix0C]. Pearson Education.

Inc., T. L. (2026). Tailwind CSS - Rapidly build modern websites without ever leaving your HTML. *Tailwind CSS is a utility-first CSS framework for rapidly building modern websites without ever leaving your HTML.* Verfügbar 20. Januar 2026 unter https://tailwindcss.com/

International Press Telecommunications Council. (2025a). *About IPTC.* https://iptc.org/about-iptc/

International Press Telecommunications Council. (2025b). *Information Interchange Model (IIM).* https://iptc.org/standards/iim/

International Press Telecommunications Council. (2025c). *IPTC Photo Metadata Standard.* https://iptc.org/standards/photo-metadata/iptc-standard/

International Telecommunication Union. (1992). *Information Technology — Digital Compression and Coding of Continuous-Tone Still Images: Requirements and Guidelines* (Recommendation ITU-T T.81 (JPEG)). ITU-T. https://www.w3.org/Graphics/JPEG/itu-t81.pdf

IPTC Photo Metadata Working Group. (2024, November). *IPTC Photo Metadata Standard 2024.1.* Verfügbar 29. August 2025 unter https://iptc.org/std/photometadata/specification/IPTC-PhotoMetadata-2024.1.html

Kulkarni, V., & Reddy, S. (2003). Separation of concerns in model-driven development. *IEEE Software*, *20*(5), 64–69. https://doi.org/10.1109/MS.2003.1231154

Mani, R. G., Parthasarathy, R., Eswaran, S., & Honnavalli, P. (2022). A Survey on Digital Image Forensics: Metadata and Image forgeries. https://api.semanticscholar.org/CorpusID:249335675

Riley, J. (2017). *Understanding metadata: what is metadata, and what is it for.* National Information Standards Organization.

Schaar, P. (2010). Privacy by Design. *Identity in the Information Society*, *3*(2), 267–274. https://doi.org/10.1007/s12394-010-0055-x

Schatten, A. (2010). *Best Practice Software-Engineering: Eine Praxiserprobte Zusammenstellung Von Komponentenorientierten Konzepten, Methoden und Werkzeugen.* Spektrum Akademischer Verlag GmbH.

Smith, K. R., Saunders, S., & Kejser, U. B. (2014). Making the Case for Embedded Metadata in Digital Images. *Archiving Conference*, *11*(1), 52–57. https://doi.org/10.2352/issn.2168-3204.2014.11.1.art00012

Sommerville, I. (2016). *Software engineering* (Tenth edition). Pearson.

Tesic, J. (2005). Metadata practices for consumer photos. *IEEE MultiMedia*, *12*(3), 86–92. https://doi.org/10.1109/MMUL.2005.50

Tkachenko, O., Chechet, A., Chernykh, M., Bunas, S., & Jatkiewicz, P. (2025). Scalable Front-End Architecture: Building for Growth and Sustainability. *Informatica*, *49*(1). https://doi.org/10.31449/inf.v49i1.6304

Verhoeven, G., Wieser, M., & Carloni, M. (2024). GRAPHIS—Visualise, Draw, Annotate, and Save Image Regions in Graffiti Photos. https://doi.org/10.48619/indigo.v0i0.977

Vom Brocke, J., Hevner, A., & Maedche, A. (Hrsg.). (2020). *Design Science Research. Cases.* Springer International Publishing. https://doi.org/10.1007/978-3-030-46781-4

You, E. (2025). Vue.js. *Vue.js - The Progressive JavaScript Framework.* Verfügbar 20. Januar 2026 unter https://vuejs.org/

Zaidan, A. J., & Suyatno, D. F. (2025). Rendering Performance Analysis of Astro JS, Next JS, Nuxt JS, and SvelteKit Frameworks Using Google Lighthouse, PageSpeed Insight, and JMeter. *6*(1). https://doi.org/https://doi.org/10.26740/jeisbi.v6i1

Zeng, M. L., & Qin, J. (2020, Juni). *Metadata* [Google-Books-ID: Y5W1EAAAQBAJ]. American Library Association.

A. Anhang

A.1. Code-Repository URL

https://github.com/fegbert/iptc-web-editor/tree/d5326f30bf9778324fa402768ec381686ea1b82e

A.2. Quellcode: *extractIIMBlockFromJPEG*

```
function extractIIMBlockFromJPEG(buffer: Uint8Array): Uint8Array {
  let offset = 0

  while (offset < buffer.length - 1) {
    // Check if current bytes match the APP13 segment - It's two bytes
         long, so we check two bytes at a time
    if (buffer[offset] === APP13_SEGMENT[0] && buffer[offset + 1] ===
        APP13_SEGMENT[1]) {
      const length = calculateSegmentLength(buffer, offset)
      const start = offset + 4 // Skipping the marker and length bytes
      const end = start + length

      const segment = buffer.subarray(start, end)

      if (!isSegmentMatchingPhotoshopHeader(segment)) {
        // If the segment does not match the Photoshop header,
            continue searching
        offset += length + 2 // Move past the current segment
        continue
      }

      return searchSegmentForImageResourceBlocks(segment,
          PHOTOSHOP_HEADER.length)
    }
    offset++
  }
  throw new Error('No␣IPTC-IIM␣block␣found␣in␣the␣JPEG␣file.')
}
```

A.3. Quellcode: *searchSegmentForImageResourceBlocks*

```
function searchSegmentForImageResourceBlocks(segment: Uint8Array,
    offset: number): Uint8Array {
  let position = offset
  // Check if the segment is long enough to contain an IRB
  while (position + 10 < segment.length) {
    if (isSegmentMatchingSignature(segment, position)) {
      // Extract the resource ID, name length and data size

      // Combining bytes to form the resource ID
      const resourceId = (segment[position + 4] << 8) | segment[
          position + 5]

      // The name length is stored in the 6th byte after the signature
      const nameLength = segment[position + 6]

      // If the name length is odd, we need to add a padding to make
          it even
      const nameEnd = position + 7 + nameLength + (nameLength % 2 ===
          0 ? 1 : 0)

      // Add the next 4 bytes together to get the data size
      const dataSize = (segment[nameEnd] << 24) | (segment[nameEnd +
          1] << 16) | (segment[nameEnd + 2] << 8) | segment[nameEnd +
          3]

      const dataStart = nameEnd + 4
      const dataEnd = dataStart + dataSize

      if (resourceId === IPTC_RESOURCE_ID) {
        // Found the IPTC-IIM block, return it
        return segment.subarray(dataStart, dataEnd)
      }

      // If dataEnd is odd, we need to add padding to make it even
      position = dataEnd + (dataEnd % 2)
    }
    else {
      position++
    }
  }
  throw new Error('No␣IPTC-IIM␣block␣was␣found␣in␣the␣segment')
}
```

A.4. Quellcode: *parseIIM*

```
export function parseIIM(buffer: Uint8Array): Record<string, any> {
  const iimBlock = extractIIMBlockFromJPEG(buffer)

  const result: Record<string, string | string[]> = {}
  const view = new DataView(iimBlock.buffer, iimBlock.byteOffset,
      iimBlock.byteLength)

  let offset = 0

  while (offset < iimBlock.length) {
    if (iimBlock[offset] !== IPTC_TAG_MARKER) {
      offset++
      continue
    }

    if (offset + 5 > iimBlock.length) {
      break
    }

    const record = iimBlock[offset + 1]
    const dataset = iimBlock[offset + 2]
    const tag = `${record}:${dataset.toString().padStart(2, '0')}`

    // Parse data length: support standard (2-byte) and extended-
        length descriptor
    const octet4 = iimBlock[offset + 3]
    const octet5 = iimBlock[offset + 4]

    let dataLength = 0
    let dataStart = 0

    // Extended-length flag is bit 7 of octet 4
    const isExtended = (octet4 & 0x80) !== 0
    if (isExtended) {
      // Length Descriptor length is the 15 LSB across octet4 (lower 7
          bits) and octet5 taken together
      const descriptorLength = ((octet4 & 0x7F) << 8) | octet5
      dataStart = offset + 5 + descriptorLength

      const descriptorEnd = offset + 5 + descriptorLength
      if (descriptorEnd > iimBlock.length) {
        break
      }

      // Parse actual data length from descriptor bytes (big-endian
          integer)
      dataLength = 0
      for (let i = 0; i < descriptorLength; i++) {
        dataLength = (dataLength << 8) | iimBlock[offset + 5 + i]
```

```
      }
    }
    else {
      // Standard 2-byte big-endian length
      dataLength = view.getUint16(offset + 3)
      dataStart = offset + 5
    }

    const end = dataStart + dataLength
    if (end > iimBlock.length) {
      break
    }

    const value = new TextDecoder().decode(iimBlock.subarray(dataStart
        , end))

    if (tag in result) {
      const existing = result[tag]
      result[tag] = Array.isArray(existing)
        ? [...existing, value]
        : [existing as string, value]
    }
    else {
      result[tag] = value
    }

    offset = end
  }

  return result
}
```

A.5. Quellcode: *writeToJPEG*

```
export async function writeToJPEG(image: File | Uint8Array, data:
    Record<string, string>, path?: string, fileHandle?:
    FileSystemFileHandle, name?: string) {
  const jpegBuffer = image instanceof File ? new Uint8Array(await
      image.arrayBuffer()) : image
  if (jpegBuffer[0] !== JPEG_SOI_MARKER[0] || jpegBuffer[1] !==
      JPEG_SOI_MARKER[1]) {
    throw new Error('Invalid␣JPEG␣file:␣does␣not␣start␣with␣SOI␣marker
        ')
  }

  let offset = 2
  let insertBefore = 2
```

```
let insertAfter: number | undefined

while (
  offset + 4 < jpegBuffer.length
  && jpegBuffer[offset] === 0xFF
  && jpegBuffer[offset + 1] >= 0xE0
  && jpegBuffer[offset + 1] < 0xF0
) {
  const segmentType = jpegBuffer[offset + 1]
  const segmentLength = calculateSegmentLength(jpegBuffer, offset)

  const segmentStart = offset
  const segmentEnd = offset + segmentLength + 2

  if (segmentEnd > jpegBuffer.length) {
    throw new Error('Invalid␣JPEG␣file:␣segment␣length␣exceeds␣
        buffer␣size')
  }

  if (segmentType === APP13_SEGMENT[1]) {
    const payload = jpegBuffer.subarray(offset + 4, segmentEnd)

    if (isSegmentMatchingPhotoshopHeader(payload)) {
      insertBefore = segmentStart
      insertAfter = segmentEnd
      break
    }
  }

  insertBefore = segmentEnd
  offset = segmentEnd
}

const before = jpegBuffer.subarray(0, insertBefore)
const after = insertAfter ? jpegBuffer.subarray(insertAfter) :
    jpegBuffer.subarray(insertBefore)

const encodedData = encodeData(data)
const irbData = wrapDataInIRB(encodedData)
const app13Data = wrapDataInAPP13(irbData)

const output = new Uint8Array(before.length + app13Data.length +
    after.length)
output.set(before, 0)
output.set(app13Data, before.length)
output.set(after, before.length + app13Data.length)

// 1. If path is provided, write the file at the specified path
if (path) {
  try {
```

```
      fs.writeFileSync(path, output)
    }
    catch (error) {
      console.error('Error␣writing␣JPEG␣file:', error)
    }
  }

  // 2. If fileHandle is provided, use it to write the file
  else if (fileHandle) {
    try {
      const writeable = await fileHandle.createWritable()
      await writeable.write(output)
      await writeable.close()
    }
    catch (error) {
      console.error('Error␣writing␣JPEG␣file␣using␣file␣handle:',
          error)
    }
  }

  // 3. If neither is provided, download the file
  else {
    const blob = new Blob([output], { type: image instanceof File ?
        image.type : 'image/jpeg' })
    try {
      await fileSave(blob, {
        fileName: name ?? (image instanceof File ? image.name : '
            unnamed.jpg'),
        extensions: ['.jpg', '.jpeg'],
      }, undefined)
    }
    catch (error) {
      console.warn('File␣save␣using␣browser-fs-access␣failed.␣Trying␣
          to␣download␣manually...')
      console.error('Error␣saving␣file:', error)
      if (typeof window !== 'undefined' && typeof document !== '
          undefined') {
        const url = URL.createObjectURL(blob)
        const linkElement = document.createElement('a')
        linkElement.href = url
        linkElement.download = name ?? (image instanceof File ? image.
            name : 'unnamed.jpg')
        document.body.appendChild(linkElement)
        linkElement.click()
        linkElement.remove()
        setTimeout(() => URL.revokeObjectURL(url), 1000)
      }
    }
  }
}
```

A.6. Quellcode: *Dockerfile*

```
# Multi-stage Dockerfile for IPTC Web Editor (Nuxt.js + pnpm workspace
    )

# Stage 1: Build
FROM node:24-alpine AS builder

# Install pnpm
RUN corepack enable && corepack prepare pnpm@10.12.4 --activate

WORKDIR /editor

# Copy workspace configuration and app package files
COPY pnpm-workspace.yaml pnpm-lock.yaml ./
COPY app/package.json app/pnpm-lock.yaml ./app/

# Copy parser module (needed for prepare script during install)
COPY parser ./parser

# Install dependencies (parser builds via prepare script)
RUN pnpm install --frozen-lockfile

# Copy app module
COPY app ./app

# Build Nuxt app (parser builds automatically via prepare script)
RUN pnpm -C app build

# Stage 2: Production runtime
FROM node:24-alpine AS runner

RUN corepack enable && corepack prepare pnpm@10.12.4 --activate

WORKDIR /editor

# Set NODE_ENV to production
ENV NODE_ENV=production

# Copy workspace configuration
COPY --from=builder /editor/pnpm-workspace.yaml ./
COPY --from=builder /editor/pnpm-lock.yaml ./

# Copy parser package (built)
COPY --from=builder /editor/parser/package.json ./parser/
COPY --from=builder /editor/parser/dist ./parser/dist

# Copy app package.json and built files
COPY --from=builder /editor/app/package.json ./app/
COPY --from=builder /editor/app/.output ./app/.output
```

```
# Install only production dependencies
RUN pnpm install --frozen-lockfile --prod --ignore-scripts

# Expose port 3000
EXPOSE 3000

# Set working directory to app
WORKDIR /editor/app

# Start the Nuxt application
CMD ["pnpm", "start"]
```

Erklärung

Ich versichere, die von mir vorgelegte Arbeit selbstständig verfasst zu haben. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder nicht veröffentlichten Arbeiten anderer oder der Verfasserin/des Verfassers selbst entnommen sind, habe ich als entnommen kenntlich gemacht. Sämtliche Quellen und Hilfsmittel, die ich für die Arbeit benutzt habe, sind angegeben. Die Arbeit hat mit gleichem Inhalt bzw. in wesentlichen Teilen noch keiner anderen Prüfungsbehörde vorgelegen.

Anmerkung: In einigen Studiengängen findet sich die Erklärung unmittelbar hinter dem Deckblatt der Arbeit.

Engelskirchen, 30.01.2026

Ort, Datum

Unterschrift